

TWO INDEPENDENT LANES • SAME BYTES • READ-ONLY

METHODS AUDIT • vs THE BLOCKMACHINE REFERENCE

Every balance we show, checked against a second road.

BlockMachine published a from-scratch reference for reading complete Bittensor account balances straight from chain state (blockmachine.io/blog/checking-bittensor-account-balances). We ran their lanes and ours **side by side, live, on the same accounts** — the EVM lane the app ships versus raw substrate storage and runtime APIs. Result: **agreement to the byte**, one subtle semantic difference now pinned, and one forward flag banked before it could ever bite.

VERIFIED

TEST ACCOUNTS = OUR OWN AGENTS' TBAs • EVERY NUMBER REPRODUCIBLE • SPIKE: `spike/balance-crosscheck.js`

FINDING 1 The 500-RAO constant — spendable vs raw free

Our lane (`eth_getBalance`) and theirs (`System.Account.free` via `state_getStorage`) disagreed by **exactly 500 RAO on both accounts** — the existential deposit. Frontier's EVM lane reports the **reducible (spendable)** balance; raw substrate free includes the deposit. Neither lane is wrong — they answer different questions. **When 8004 shows a balance, it shows what the account can spend.**

ACCOUNT	OURS • ETH_GETBALANCE	THEIRS • SYSTEM.ACCOUNT.FREE	Δ
Agent #1 TBA 0xd42c...6778	43,288,895 RAO (×1e9 wei)	43,289,395 RAO	500 RAO — the ED
Agent #2 TBA 0xd022...b1Ea	99,500 RAO (×1e9 wei)	100,000 RAO	500 RAO — the ED

the Δ is CONSTANT and identical across unrelated accounts — a semantic difference (τ0.00000005), not drift. reserved = 0 and frozen = 0 on both, so free-only displays lose nothing today.

FINDING 2 Their u64 caveat — proven, not just believed

The reference warns that Bittensor's `System.Account` balances are **u64, not u128 as on Polkadot mainnet** — mismatched width assumptions decode to silent garbage. We proved it from the blob itself: the returned `AccountInfo` is **56 bytes** on both accounts (u64 layout); Polkadot's u128 layout would be 80. Decode offsets on chain 964: free @16 • reserved @24 • frozen @32, all u64 LE.

FINDING 3 Stake lanes agree — including on emptiness

Their stake lane is a runtime-API sweep (`StakeInfoRuntimeApi_get_stake_info_for_coldkey`, a compact-SCALE vector of **9-field** `StakeInfo` structs — truncate the struct and the decoder desyncs, exactly as they warn). Ours is the 0x805 precompile `getStake(hotkey, blake2_256("evm:" || H160), netuid)` per known position. Run live on Agent #1's TBA — which registered SN116 UID 86 and later harvested its alpha back — **both lanes returned empty/zero**: chain-true, and in agreement.

METHOD NOTE — WHEN TO USE WHICH LANE

The sweep is the **stronger read for total AUM**: it finds every position under a coldkey, including ones you didn't know to ask about. Our precompile reads are **position-scoped by construction** — exact, cheap, but only as complete as the position list. Their `SwapRuntimeApi_current_alpha_price` (netuid as u16-LE) also verified live — SN116 priced at **0.040524601 τ/a** at check time — a third, indexer-free lane for valuing alpha.

⚠ FORWARD FLAG — BANKED BEFORE IT BITES

Reserved balance is 0 on every account we touch today — but proxy deposits reserve TAO. The moment the 0x80b proxy precompile enters our stack (`createPureProxy/addProxy` both take deposits), a free-only display will silently understate holdings. The BM total is `free + reserved + root stake +  $\Sigma(\text{alpha} \times \text{price})$` ; ours must grow the reserved term the same day proxies land.

Appendix — reproduce it

```
spike spike/balance-crosscheck.js (read-only: eth_getBalance + eth_call vs state_getStorage + state_call)
evm lane lite.chain.opentensor.ai · substrate lane entrypoint-finney.opentensor.ai
storage key twox128("System") || twox128("Account") || blake2_128(acct) || acct — twox128 pure-JS, self-tested vs
twox128("System") = 26aa394e...
coldkey mapping acct = blake2_256("evm:" || H160) — the June-20 footgun: raw-H160-as-bytes32 reads silently return zero
accounts 0xd42c953c77fae46e3a05262f7b3fbe6e64666778 (agent #1) · 0xd022E0EF3B1916368Eb649660CF0edf3DdFfb1Ea (agent #2)
reference blockmachine.io/blog/checking-bittensor-account-balances · runtime node-subtensor/v424.1 · anchors
docs/verification-anchors.md §10
```